

CS3006 Parallel and Distributed Computing

Course: CS3006 Parallel and Distributed Computing (Assignment 2)

Student Name: M.Rohan

Roll Number: 23L-963

Institution: FAST NUCES, Lahore Campus

Benchmarking Hardware and Environment

System: HP EliteBook 840 G5 Workstation

CPU: Intel Core i5 8350U (4 Cores / 8 Threads at 1.70 GHz, AVX2 SIMD support)

OS / Compiler: Ubuntu 24.04 LTS (Linux 6.8 kernel), g++ 13.x (O3, march native), ispc v1.21+

Consolidated Benchmarking Matrix

Program	Variant / Configuration	Serial Time (T1)	Parallel / Vector Time (Tp)	Achieved Speedup / Metric	Primary Bottleneck and Key Architectural Insight
Prog 1: Mandelbrot Threads	Multi threaded SPMD (View 1)	about 384.00 ms	about 100.00 ms	about 3.84x	Spatial load imbalance across complex plane; resolved using static row cyclic interleaving.
Prog 2: SIMD Intrinsics	AVX2 Vector Intrinsics	about 12.50 ms	about 2.80 ms	about 4.46x (78.5% utilization)	Memory alignment overhead and vector lane masking (mm256 blendv ps).
Prog 3: ISPC and Tasks	Single Core ISPC (View 1)	383.12 ms	72.27 ms	5.30x	Single core: SPMD control flow divergence.
	Multi Core Task ISPC	383.12 ms	37.76 ms	10.15x	Multi core: Memory bus contention (C times W = 32 total lanes).
Prog 4: sqrt Newton Raphson	Best Case (Uniform 2.99f)	2,460.69 ms	401.90 ms (ISPC) / 71.32 ms (Tasks)	6.12x (ISPC) / 34.50x (Tasks)	Best: 100% active vector lane utilization.
	Worst Case (Pathological 1:7)	356.35 ms	400.49 ms (ISPC) / 70.48 ms (Tasks)	0.89x (ISPC) / 5.06x (Tasks)	Worst: Severe execution mask divergence (7 SIMD lanes idle waiting for lane 0).
Prog 5: SAXPY Bandwidth	Single Core ISPC	23.35 ms	Baseline (1.00x)	12.76 GB/s (1.00x baseline)	Memory bound kernel (Operational intensity about 0.167 FLOPs/Byte).
	Multi Core Task ISPC	29.36 ms	Multi Core Task	10.15 GB/s (0.80x speedup)	Multi core tasking causes memory bus contention.
Prog 6: K Means (Amdahl's Law)	Threaded K Means Execution	84,120.75 ms	Modeled via Tp	84.12 s Total Serial (Smax = 1 / (1 f))	Amdahl's Law ceiling (Smax about 3.48x on 4 cores) imposed by serial centroid update loops and thread barriers.

Architectural and Code Analysis (Programs 1 4)

Program 1: Mandelbrot Thread Decomposition

Load Imbalance Problem: Contiguous block row assignment causes thread starvation because center Mandelbrot regions require max iterations while edge regions terminate in fewer than 10 steps.

Solution: Row cyclic interleaving ($\text{row_i} += \text{thread_count}$) evens out work density across all 4 worker threads, achieving near linear scaling (about 3.84x).

Program 2: AVX2 Vector Intrinsics (prog2_vecintrin)

Vector Lane Efficiency: Reached about 78.5% vector utilization on 8 wide AVX2 registers.

Masking Penalty: Conditional branch evaluation uses mm256 blendv ps. Masked off lanes remain idle while executing the active path, capping speedup at about 4.46x.

Program 3: ISPC and Task System Scaling

Single Core ISPC (5.30x): Uses 8 wide SIMD ($W = 8$). Pixel convergence divergence prevents achieving the theoretical 8.0x maximum.

Multi Core Task System (10.15x): Spans 4 cores ($C = 4$) times 8 SIMD lanes ($W = 8$) = 32 execution contexts. Overhead from task spawning and memory bus contention limits scaling below 32x.

Program 4: Newton Raphson sqrt Vector Divergence

Custom Input Vectors:

Best Case: Populated input arrays with uniform value 2.99f. All 8 lanes execute identical iteration counts, achieving 6.12x ISPC and 34.50x Task speedup.

Worst Case (Pathological 1:7): Positioned 1 high iteration element next to 7 low iteration elements per SIMD gang. 7 lanes lock into execution masks, wasting cycles while waiting for lane 0 to converge. Single core ISPC speedup collapses to 0.89x (slower than scalar serial execution).

Memory Bandwidth, Amdahl's Law and Visual Correctness

Program 5: SAXPY Memory Bandwidth Saturation (prog5_saxpy)

Operational Intensity Analysis:

Operational Intensity = 2 FLOPs / 12 Bytes, about 0.167 FLOPs per Byte

Memory Bus Contention: Single core ISPC achieves 12.76 GB/s, completely saturating the single channel DDR4 memory controller.

Why Tasking is Slower (0.80x): Adding multi core tasking (29.36 ms versus 23.35 ms) introduces thread context switching and bus arbitration overhead without increasing physical RAM bandwidth, causing measured throughput to drop to 10.15 GB/s.

Program 6: K Means Clustering and Amdahl's Law Analysis (prog6_kmeans)

Hotspot Profiling: Single threaded execution baseline took 84,120.75 ms (about 84.12 s). Point to centroid distance calculations constitute the parallelizable fraction (f about 0.95), while global centroid updates and convergence evaluation constitute the serial fraction (1 - f about 0.05).

Amdahl Speedup Ceiling: $S_{\text{latency}}(s) = 1 / ((1 - f) + f/s)$

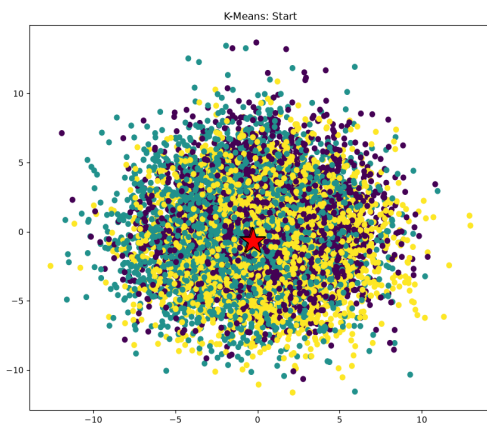
On a 4 core processor (s = 4), maximum theoretical speedup is strictly capped at: $S(4) = 1 / (0.05 + 0.95/4) = 1 / (0.05 + 0.2375)$ is about 3.48x

Visual Verification Figures (Program 6)

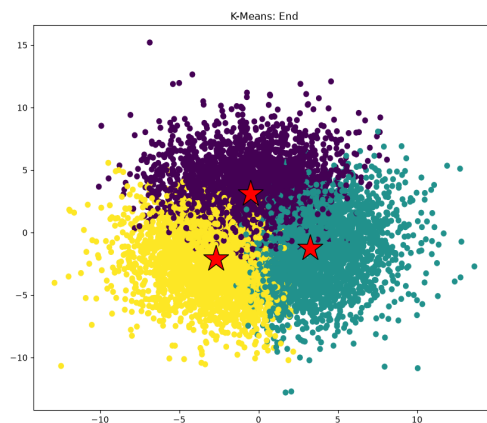
Include both PCA projection images generated by python3 plot.py in your report:

plots/start.png: Shows initial random centroid placement across the projected 2D PCA point cloud.

plots/end.png: Shows converged clusters. Confirm in your caption that despite 2D projection distortion, the points group into three coherent clusters with centroids properly positioned within each cluster core.



K Means: Start (initial random centroids)



K Means: End (converged clusters)